



C语言三周入门手册

一、教学目标

本教程面向零基础的同学，希望能帮助大家在 三周时间内系统入门 C 语言，理解从解释型语言到编译型语言的差异，掌握 C 语言的基础语法、指针与数组、内存管理、模块化编程等核心知识。

通过课程学习，学生将能够：

- 理解 C 程序的编译与执行流程；
- 掌握常用**数据类型**、**控制语句**、**函数**、**数组与指针**；
- 学会使用 **GNU 工具链**（gcc、gdb、make、CMake）进行项目开发；
- 能够编写并管理多文件工程；
- 通过综合项目实践，初步形成“工程化编程思维”。

最终成果包括：

- ☒ 完成一个控制台版“学生成绩管理系统”；
-  扩展挑战：基于 C 语言实现一个控制台版的“Flappy Bird”小游戏。

【注】本教程的功能仅在于入门学习，适合初学者快速上手 C 语言编程。不能够涵盖 C 语言的所有高级特性和应用场景，**不能**替代系统的编程课程。

推荐学习资料

更系统的学习资料请参考以下资源：

在线教程：

- [C 语言官方文档](#)：涵盖 C 语言标准规范，是最权威的参考资料。
- [菜鸟教程 C 语言教程](#)：以简洁的语言讲解 C 语言基础语法，搭配实例代码，适合零基础入门。

书籍：

- 《C 程序设计语言（第 2 版）》：由 C 语言设计者编写，是 C 语言领域的经典教材，内容精炼且权威，适合深入理解 C 语言的设计思想。
- 《C Primer Plus（第 6 版）》：内容详实，涵盖 C 语言的各个方面，配有大量实例和练习题，适合系统学习和实践。

- 《C 和指针》：聚焦 C 语言的指针特性，通过大量实例解析指针在内存操作、函数调用等场景的应用，帮助攻克 C 语言学习中的难点。

视频课程：

- B 站“浙江大学翁恺 C 语言”课程（BV 号：**BV1dr4y1n7vA**）：由知名教授授课，注重基础概念的讲解与编程思维的培养，适合打牢 C 语言基础。
- B 站“黑马程序员 C 语言教程”（BV 号：**BV1Xa4y1k7LU**）：从 C 语言基础语法到标准库，内容系统且贴近实战，适合零基础学习者循序渐进掌握 C 语言。

以上资源均适合零基础学习者，能够帮助你系统掌握 C 语言编程的核心知识与技能。建议结合多种形式的学习资料，选一个你自己**能接受学进去**的作为主要学习材料，其他的作为辅助，理论与实践相结合，提升编程能力。

二、教程安排

教程共 **21 天**，分为三周进行。

- **第一周**：重点掌握 **C 语言基础语法与编译型思维**，理解 C 程序结构、变量与数据类型、运算符、条件语句与循环语句。
- **第二周**：重点学习 **数组、指针、字符串、函数、内存管理**，建立内存视角，理解函数参数传递机制。
- **第三周**：重点学习 **模块化编程、头文件、文件操作、Makefile 与 CMake 工程化构建**，并完成综合项目“学生成绩管理系统”，扩展项目为控制台 Flappy Bird。

本教程的每天学习内容包括：

- **知识点**：当天需要掌握的核心概念与语法。
- **小任务**：通过编写代码完成的小练习，巩固当天所学内容。

【注】本教程提供的小任务和综合项目均为参考示例，学习者可根据自身兴趣和需求进行调整和扩展。**不强制完成**，完成情况**不计入最终考核评分**。

第一周安排（Day 1–7）

天数	主题	知识点	小任务
D1	C语言入门与Hello World	C语言历史、编译型 vs 解释型、main函数、printf	编写 Hello World 程序
D2	数据类型与变量	int, float, char, double, 常量定义、格式化输出	输入两个数并输出和与差
D3	运算符	算术、逻辑、关系运算符，运算符优先级	编写计算器程序，完成加减乘除运算
D4	条件语句	if, if-else, switch, 嵌套分支	输入成绩并输出等级（优秀/良好/及格/不及格）
D5	循环语句	for, while, do-while, break/continue	打印 1–100 的偶数和奇数
D6	函数基础	函数定义与调用、参数传递、返回值、作用域	编写函数实现求两数最大值
D7	周总结与练习	基础语法综合应用	综合练习：猜数字小游戏

第二周安排（Day 8–14）

天数	主题	知识点	小任务
D8	一维数组	数组定义、初始化、遍历、与 Python 列表对比	编写数组求和程序
D9	二维数组	矩阵表示、多维数组访问	输出九九乘法表
D10	字符数组与字符串	字符串定义、strlen、strcpy、strcmp	输入字符串并反转输出
D11	指针基础	指针定义、取址 & 解引用 *	编写 swap 函数，使用指针交换两个变量
D12	指针与函数	数组指针、函数参数传递、字符串指针	编写函数统计数组最大值
D13	结构体与 typedef	结构体定义、嵌套与数组结合	定义学生结构体并输入信息

天数	主题	知识点	小任务
D14	周总结	综合练习	简易通讯录（数组+结构体）

第三周安排（Day 15–21）

天数	主题	知识点	小任务
D15	文件操作	文件读写、文本与二进制文件	保存成绩到文件并读取
D16	动态内存	malloc/free、sizeof、内存泄漏问题	编写动态数组求平均值
D17	头文件与模块化	函数声明、头文件分离、分文件编程	分文件实现简单函数库
D18	GNU工具链	gcc 编译、gdb 调试、make基础	编译多文件项目，单步调试
D19	Makefile与CMake	CMakeLists.txt、跨平台构建	使用 CMake 构建学生成绩系统
D20	综合项目①	学生成绩管理系统模块设计与实现	实现增删改查功能
D21	综合项目②（扩展）	控制台 Flappy Bird 小游戏	实现跳跃和障碍逻辑

三、预期成果

- 同学们能独立编写 C语言 解决简单问题
- 能熟练使用基本语法、函数、数组与指针
- 理解内存管理与模块化编程
- 掌握 GNU 工具链进行项目开发
- 完成一个完整的“学生成绩管理系统”大作业
- 挑战扩展：实现一个控制台版的“Flappy Bird”小游戏

四、每日学习内容

项目结构准备

在开始正式学习之前，我们需要先准备好项目的基本目录结构，方便每天的代码和总结文档进行管理。

【注】本项目结构仅为建议，学习者可根据个人习惯进行调整，但建议保持清晰有序，便于后续复习和查阅。

操作步骤

1. 打开你的工作目录（例如 `D:\my-first-project` 或 `~/my-first-project`）。
2. 在其中新建一个文件夹：

```
my-first-project/  
└─ C 语言学习/
```

3. 在 `C 语言学习` 文件夹下，建立 `Day 1 ~ Day 21` 共 **21 个子文件夹**，用于存放每天的代码和练习：

```
C 语言学习/  
├─ Day0/  
├─ Day1/  
├─ Day2/  
├─ ...  
└─ Day21/
```

文件组织规范

- 每个 `DayX/` 文件夹下，至少包含：
 - 当天的小任务代码文件（例如源文件 `main.c`，编译后的可执行文件 `main.exe`）。
 - 如果有总结或笔记，推荐使用 **Markdown 文件**（例如 `notes.md`）来记录当天的学习要点。

例如，`Day1/` 文件夹可能长这样：

```
Day1/  
├─ main.c      # 小任务代码  
├─ main.exe    # 编译后的可执行文件  
└─ notes.md    # 知识点总结
```

提交至 git 与 GitHub

1. 将 `my-first-project` 文件夹初始化为一个 Git 仓库：

```
cd my-first-project
git init
```

2. 每天学习结束后，将当天的代码和笔记提交到 Git 仓库：

```
git add .
git commit -m "完成 DayX 学习任务"
```

3. 如果有远程仓库（例如 GitHub），可以将本地仓库与远程仓库关联，并推送代码：

```
git remote add origin <你的远程仓库地址>
git push -u origin main
```

【注】推荐将每日的进度提交到 Git 仓库和 GitHub，养成良好的版本管理习惯，便于后续复习和查阅。

推荐做法

- **代码文件**：将每天的小任务实现写在 `main.c` 或拆分成多个小文件。
- **笔记文件**：使用 Markdown 简洁总结：
 - 当天学习的语法/概念
 - 小任务的实现思路
 - 遇到的问题与解决办法

这样到第 21 天结束时，你将拥有一个完整的 **学习代码库 + 学习日志**，方便复习与回顾。

Day 1: C语言入门与Hello World

1.1 基本知识点

- C 语言的历史与特点
- 编译型语言 vs 解释型语言的差异
- C 程序的基本结构（main 函数、头文件、语句）
- `printf()` 函数的基本用法
- 编译与运行 C 程序

1.2 学习资料

- 菜鸟教程：
 - [C 语言教程](#)

- [C 程序结构](#)
- C 官方文档：[ISO C 标准](#)

✅ 1.3 应知应会 Checklist

- ☐ 理解编译型语言的执行流程
- ☐ 掌握 C 程序的基本结构
- ☐ 会使用 gcc 编译并运行 C 程序
- ☐ 能编写并输出 "Hello, World!"

📄 1.4 小任务

编写第一个 C 程序，输出 "Hello, World!"：

```
// hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

在终端中编译并运行：

```
gcc hello.c -o hello
./hello
```

Day 2：数据类型与变量

■ 2.1 基本知识点

- 基本数据类型：int、float、double、char
- 变量的声明与初始化
- 常量定义：const 和 #define
- 格式化输入输出：printf() 和 scanf()
- 数据类型的大小（sizeof 运算符）

2.2 学习资料

- 菜鸟教程：
 - [C 数据类型](#)
 - [C 变量](#)
 - [C 常量](#)
 - [C 输入 & 输出](#)

2.3 应知应会 Checklist

- ☐ 理解不同数据类型的用途和区别
- ☐ 能正确声明和初始化变量
- ☐ 掌握 `printf()` 的格式化输出（`%d`, `%f`, `%c` 等）
- ☐ 掌握 `scanf()` 的使用方法
- ☐ 了解各数据类型占用的字节数

2.4 小任务

编写程序，输入两个整数，输出它们的和与差：

```
#include <stdio.h>

int main() {
    int a, b;
    printf("请输入两个整数: ");
    scanf("%d %d", &a, &b);

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);

    return 0;
}
```

Day 3: 运算符

3.1 基本知识点

- 算术运算符：`+`、`-`、`*`、`/`、`%`
- 关系运算符：`==`、`!=`、`>`、`<`、`>=`、`<=`

- 逻辑运算符： && 、 || 、 !
- 赋值运算符： = 、 += 、 -= 、 *= 、 /=
- 自增自减运算符： ++ 、 --
- 运算符优先级与结合性

3.2 学习资料

- 菜鸟教程：
 - [C 运算符](#)
 - [C 运算符优先级](#)

3.3 应知应会 Checklist

- ☐ 掌握算术运算符的使用
- ☐ 理解整数除法与取模运算
- ☐ 能使用逻辑运算符进行条件判断
- ☐ 理解自增自减运算符的前缀和后缀差异
- ☐ 了解运算符优先级

3.4 小任务

编写一个简单的计算器程序，实现加减乘除运算：

```
#include <stdio.h>

int main() {
    int a, b;
    char op;

    printf("请输入表达式 (例如: 5 + 3): ");
    scanf("%d %c %d", &a, &op, &b);

    switch(op) {
        case '+': printf("结果: %d\n", a + b); break;
        case '-': printf("结果: %d\n", a - b); break;
        case '*': printf("结果: %d\n", a * b); break;
        case '/':
            if(b != 0) printf("结果: %d\n", a / b);
            else printf("错误: 除数不能为0\n");
            break;
        default: printf("无效运算符\n");
    }

    return 0;
}
```

Day 4: 条件语句

4.1 基本知识点

- if 语句的基本语法
- if-else 和 if-else if-else 多分支结构
- switch-case 语句
- 条件表达式 (三元运算符): ? :
- 嵌套条件语句

4.2 学习资料

- 菜鸟教程:
 - [C 判断](#)
 - [C if 语句](#)
 - [C switch 语句](#)

✓ 4.3 应知应会 Checklist

- ☐ 能编写单分支和多分支条件语句
- ☐ 理解 switch-case 的使用场景
- ☐ 掌握 break 在 switch 中的作用
- ☐ 能使用三元运算符简化简单判断
- ☐ 理解条件语句的嵌套

📄 4.4 小任务

编写程序，输入学生成绩，输出对应等级：

- 90-100：优秀
- 70-89：良好
- 60-69：及格
- 0-59：不及格

```
#include <stdio.h>
```

```
int main() {  
    int score;  
    printf("请输入成绩: ");  
    scanf("%d", &score);  
  
    if(score >= 90 && score <= 100) {  
        printf("等级: 优秀\n");  
    } else if(score >= 70) {  
        printf("等级: 良好\n");  
    } else if(score >= 60) {  
        printf("等级: 及格\n");  
    } else if(score >= 0) {  
        printf("等级: 不及格\n");  
    } else {  
        printf("无效成绩\n");  
    }  
  
    return 0;  
}
```

Day 5: 循环语句

5.1 基本知识点

- for 循环的语法与执行流程
- while 循环
- do-while 循环
- break 和 continue 语句
- 循环的嵌套
- 无限循环与循环控制

5.2 学习资料

- 菜鸟教程：
 - [C 循环](#)
 - [C for 循环](#)
 - [C while 循环](#)
 - [C do...while 循环](#)

5.3 应知应会 Checklist

- ☐ 理解三种循环的区别和使用场景
- ☐ 能使用 for 循环遍历指定范围
- ☐ 掌握 break 和 continue 的用法
- ☐ 能编写嵌套循环
- ☐ 理解循环的执行流程

5.4 小任务

编写程序，分别打印 1-100 的所有偶数和奇数：

```
#include <stdio.h>

int main() {
    printf("偶数: ");
    for(int i = 1; i <= 100; i++) {
        if(i % 2 == 0) {
            printf("%d ", i);
        }
    }
    printf("\n");

    printf("奇数: ");
    for(int i = 1; i <= 100; i++) {
        if(i % 2 != 0) {
            printf("%d ", i);
        }
    }
    printf("\n");

    return 0;
}
```

Day 6: 函数基础

6.1 基本知识点

- 函数的定义与声明
- 函数参数与返回值
- 函数调用过程
- 函数原型（函数声明）
- 变量的作用域（局部变量与全局变量）
- 递归函数初步

6.2 学习资料

- 菜鸟教程：
 - [C 函数](#)
 - [C 作用域规则](#)

✅ 6.3 应知应会 Checklist

- ☐ 能定义和调用函数
- ☐ 理解函数参数的传递机制
- ☐ 掌握函数返回值的使用
- ☐ 理解局部变量和全局变量的区别
- ☐ 了解函数原型的作用

📄 6.4 小任务

编写函数 `max(int a, int b)`，返回两个整数中的较大值：

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int x, y;
    printf("请输入两个整数: ");
    scanf("%d %d", &x, &y);

    printf("较大值: %d\n", max(x, y));

    return 0;
}
```

Day 7: 周总结与练习

📌 7.1 基本知识点

- 复习第一周所学内容
- 综合运用变量、运算符、条件语句、循环、函数

🔗 7.2 学习资料

- 回顾 Day 1-6 的知识点和代码
- 菜鸟教程：[C 练习实例](#)

✅ 7.3 应知应会 Checklist

- ☐ 能综合运用所学知识解决问题
- ☐ 理解程序的执行流程
- ☐ 能调试简单的程序错误

📄 7.4 小任务：猜数字游戏

编写一个猜数字小游戏：

- 程序随机生成一个 1-100 的整数
- 用户输入猜测的数字
- 程序提示"太大"、"太小"或"猜对了"
- 统计猜测次数

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(NULL)); // 初始化随机数种子
    int target = rand() % 100 + 1;
    int guess, count = 0;

    printf("猜数字游戏开始！我想了一个 1-100 的数字。\\n");

    do {
        printf("请输入你的猜测：");
        scanf("%d", &guess);
        count++;

        if(guess > target) {
            printf("太大了！\\n");
        } else if(guess < target) {
            printf("太小了！\\n");
        } else {
            printf("恭喜你猜对了！共猜了 %d 次。\\n", count);
        }
    } while(guess != target);

    return 0;
}
```

Day 8：一维数组

8.1 基本知识点

- 数组的概念与作用
- 一维数组的声明与初始化
- 数组元素的访问与遍历
- 数组的长度（使用 `sizeof`）
- C 数组与 Python 列表的对比

8.2 学习资料

- 菜鸟教程：
 - [C 数组](#)

8.3 应知应会 Checklist

- ☐ 能声明并初始化一维数组
- ☐ 能通过下标访问数组元素
- ☐ 能使用循环遍历数组
- ☐ 理解数组在内存中的连续存储
- ☐ 了解 C 数组与 Python 列表的差异

8.4 小任务

编写程序，输入 5 个整数存入数组，计算并输出它们的和与平均值：

```
#include <stdio.h>

int main() {
    int nums[5];
    int sum = 0;

    printf("请输入 5 个整数:\n");
    for(int i = 0; i < 5; i++) {
        scanf("%d", &nums[i]);
        sum += nums[i];
    }

    printf("总和: %d\n", sum);
    printf("平均值: %.2f\n", sum / 5.0);

    return 0;
}
```

Day 9: 二维数组

9.1 基本知识点

- 二维数组的声明与初始化
- 二维数组的访问（行、列索引）
- 使用嵌套循环遍历二维数组
- 矩阵的表示与运算

9.2 学习资料

- 菜鸟教程：
 - [C 多维数组](#)

9.3 应知应会 Checklist

- ☐ 能声明并初始化二维数组
- ☐ 理解二维数组的行列结构
- ☐ 能使用嵌套循环遍历二维数组
- ☐ 能用二维数组表示矩阵

9.4 小任务

编写程序，输出九九乘法表：

```
#include <stdio.h>

int main() {
    for(int i = 1; i <= 9; i++) {
        for(int j = 1; j <= i; j++) {
            printf("%d*%d=%2d ", j, i, i*j);
        }
        printf("\n");
    }

    return 0;
}
```

Day 10：字符数组与字符串

10.1 基本知识点

- 字符数组的声明与初始化
- 字符串的表示（以 `\0` 结尾）
- 字符串的输入输出（`scanf`、`gets`、`fgets`）
- 常用字符串函数：`strlen()`、`strcpy()`、`strcmp()`、`strcat()`
- 字符串与字符数组的区别

10.2 学习资料

- 菜鸟教程：
 - [C 字符串](#)

10.3 应知应会 Checklist

- ☐ 理解字符串在 C 中的存储方式
- ☐ 能声明和初始化字符数组
- ☐ 掌握常用字符串函数的使用
- ☐ 能进行字符串的输入输出
- ☐ 理解 `\0` 字符的作用

10.4 小任务

编写程序，输入一个字符串，将其反转后输出：

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    printf("请输入一个字符串：");
    fgets(str, sizeof(str), stdin);

    // 去掉换行符
    str[strcspn(str, "\n")] = '\0';

    int len = strlen(str);
    printf("反转后：");
    for(int i = len - 1; i >= 0; i--) {
        printf("%c", str[i]);
    }
    printf("\n");

    return 0;
}
```

Day 11：指针基础

11.1 基本知识点

- 指针的概念与意义
- 指针变量的声明
- 取址运算符 & 和解引用运算符 *
- 指针与变量的关系
- 指针的初始化与使用
- 空指针（NULL）

11.2 学习资料

- 菜鸟教程：
 - [C 指针](#)

- 《C 和指针》相关章节

✓ 11.3 应知应会 Checklist

- ☐ 理解指针的概念和用途
- ☐ 能声明并初始化指针变量
- ☐ 掌握 & 和 * 运算符的使用
- ☐ 理解指针与内存地址的关系
- ☐ 了解空指针的作用

11.4 小任务

编写 swap() 函数，使用指针交换两个变量的值：

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("交换前: x = %d, y = %d\n", x, y);

    swap(&x, &y);

    printf("交换后: x = %d, y = %d\n", x, y);

    return 0;
}
```

Day 12: 指针与函数

12.1 基本知识点

- 指针作为函数参数
- 通过指针修改函数外部变量
- 函数返回指针

- 指针与数组的关系
- 数组名作为指针
- 指针算术运算

12.2 学习资料

- 菜鸟教程：
 - [C 指针](#)
 - [C 函数指针与回调函数](#)

12.3 应知应会 Checklist

- ☐ 理解指针参数的传递机制
- ☐ 能通过指针在函数中修改外部变量
- ☐ 理解数组名与指针的关系
- ☐ 能使用指针遍历数组
- ☐ 了解指针的算术运算

12.4 小任务

编写函数 `findMax(int *arr, int size)`，使用指针找出数组中的最大值：

```
#include <stdio.h>

int findMax(int *arr, int size) {
    int max = arr[0];
    for(int i = 1; i < size; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

int main() {
    int nums[] = {3, 7, 2, 9, 1, 5};
    int size = sizeof(nums) / sizeof(nums[0]);

    printf("最大值: %d\n", findMax(nums, size));

    return 0;
}
```

Day 13: 结构体与 typedef

13.1 基本知识点

- 结构体的定义与声明
- 结构体变量的创建与初始化
- 访问结构体成员（. 运算符）
- 结构体指针（-> 运算符）
- 结构体数组
- 结构体嵌套
- typedef 定义类型别名

13.2 学习资料

- 菜鸟教程：
 - [C 结构体](#)
 - [C typedef](#)

13.3 应知应会 Checklist

- ☐ 能定义和使用结构体
- ☐ 理解结构体成员的访问方式
- ☐ 能使用结构体数组
- ☐ 掌握结构体指针的使用
- ☐ 能使用 typedef 简化代码

13.4 小任务

定义学生结构体，输入多个学生信息并显示：

```

#include <stdio.h>

typedef struct {
    char name[50];
    int id;
    float score;
} Student;

int main() {
    Student students[3];

    printf("请输入 3 个学生的信息:\n");
    for(int i = 0; i < 3; i++) {
        printf("学生 %d - 姓名: ", i+1);
        scanf("%s", students[i].name);
        printf("学号: ");
        scanf("%d", &students[i].id);
        printf("成绩: ");
        scanf("%f", &students[i].score);
    }

    printf("\n学生信息列表:\n");
    for(int i = 0; i < 3; i++) {
        printf("%s (学号: %d) - 成绩: %.2f\n",
            students[i].name, students[i].id, students[i].score);
    }

    return 0;
}

```

Day 14: 周总结与练习

14.1 基本知识点

- 复习第二周所学内容
- 综合运用数组、指针、字符串、动态内存

14.2 学习资料

- 回顾 Day 8-13 的知识点和代码

- 菜鸟教程：[C 练习实例](#)

✓ 14.3 应知应会 Checklist

- ☐ 能熟练使用数组和指针
- ☐ 理解内存管理的重要性
- ☐ 能编写结构化的程序

14.4 小任务：简易通讯录

编写一个简易通讯录程序，使用结构体数组存储联系人信息（姓名、电话），实现添加和显示功能：

```
#include <stdio.h>
#include <string.h>

#define MAX_CONTACTS 100

typedef struct {
    char name[50];
    char phone[20];
} Contact;

Contact contacts[MAX_CONTACTS];
int count = 0;

void addContact() {
    if(count >= MAX_CONTACTS) {
        printf("通讯录已满\n");
        return;
    }

    printf("请输入姓名: ");
    scanf("%s", contacts[count].name);
    printf("请输入电话: ");
    scanf("%s", contacts[count].phone);
    count++;
    printf("添加成功\n");
}

void showContacts() {
    printf("\n通讯录列表:\n");
    for(int i = 0; i < count; i++) {
        printf("%d. %s - %s\n", i+1, contacts[i].name, contacts[i].phone);
    }
}

int main() {
    int choice;

    while(1) {
        printf("\n1. 添加联系人\n2. 显示所有联系人\n3. 退出\n请选择: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: addContact(); break;
```

```
        case 2: showContacts(); break;
        case 3: return 0;
        default: printf("无效选择\n");
    }
}

return 0;
}
```

Day 15: 文件操作

15.1 基本知识点

- 文件的概念与分类（文本文件、二进制文件）
- 文件指针（FILE *）
- 文件打开与关闭：fopen()、fclose()
- 文件读写模式：r、w、a、rb、wb、ab
- 文本文件读写：fprintf()、fscanf()、fgets()、fputs()
- 二进制文件读写：fread()、fwrite()
- 文件位置操作：fseek()、ftell()、rewind()

15.2 学习资料

- 菜鸟教程：
 - [C 文件读写](#)

15.3 应知应会 Checklist

- ☐ 理解文件操作的基本流程
- ☐ 能打开和关闭文件
- ☐ 掌握文本文件的读写
- ☐ 了解二进制文件的读写
- ☐ 能处理文件操作中的错误

15.4 小任务

编写程序，将学生成绩保存到文件，然后读取并显示：

```

#include <stdio.h>

int main() {
    // 写入文件
    FILE *fp = fopen("scores.txt", "w");
    if(fp == NULL) {
        printf("文件打开失败\n");
        return 1;
    }

    fprintf(fp, "张三 85\n");
    fprintf(fp, "李四 90\n");
    fprintf(fp, "王五 78\n");
    fclose(fp);

    // 读取文件
    fp = fopen("scores.txt", "r");
    if(fp == NULL) {
        printf("文件打开失败\n");
        return 1;
    }

    char name[50];
    int score;
    printf("学生成绩:\n");
    while(fscanf(fp, "%s %d", name, &score) != EOF) {
        printf("%s: %d\n", name, score);
    }
    fclose(fp);

    return 0;
}

```

Day 16: 动态内存管理

16.1 基本知识点

- 静态内存 vs 动态内存
- malloc() 函数：分配内存
- calloc() 函数：分配并初始化内存

- `realloc()` 函数：重新分配内存
- `free()` 函数：释放内存
- `sizeof` 运算符的使用
- 内存泄漏问题及其避免

16.2 学习资料

- 菜鸟教程：
 - [C 内存管理](#)

16.3 应知应会 Checklist

- ☐ 理解动态内存分配的必要性
- ☐ 能使用 `malloc()` 分配内存
- ☐ 掌握 `free()` 释放内存的重要性
- ☐ 理解内存泄漏的概念
- ☐ 能编写使用动态数组的程序

16.4 小任务

编写程序，动态分配数组，输入 n 个数字，计算平均值：

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("请输入数字个数: ");
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));
    if(arr == NULL) {
        printf("内存分配失败\n");
        return 1;
    }

    printf("请输入 %d 个数字:\n", n);
    int sum = 0;
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        sum += arr[i];
    }

    printf("平均值: %.2f\n", sum / (double)n);

    free(arr); // 释放内存

    return 0;
}

```

Day 17: 头文件与模块化编程

17.1 基本知识点

- 头文件的作用与组成
- 函数声明与定义的分离
- #include 指令 (<> vs "")
- 头文件保护 (#ifndef 、 #define 、 #endif)
- 多文件编程的基本结构
- 分文件编译与链接

17.2 学习资料

- 菜鸟教程：
 - [C 头文件](#)
 - [C 预处理器](#)

17.3 应知应会 Checklist

- ☐ 理解头文件的作用
- ☐ 能创建自定义头文件
- ☐ 掌握头文件保护的写法
- ☐ 能将程序拆分为多个文件
- ☐ 理解编译和链接的过程

17.4 小任务

将数学计算函数分离到独立文件中：

math_utils.h:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
float divide(int a, int b);

#endif
```

math_utils.c:

```
#include "math_utils.h"

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int multiply(int a, int b) {
    return a * b;
}

float divide(int a, int b) {
    if(b != 0) return (float)a / b;
    return 0;
}
```

main.c:

```
#include <stdio.h>
#include "math_utils.h"

int main() {
    printf("5 + 3 = %d\n", add(5, 3));
    printf("5 - 3 = %d\n", subtract(5, 3));
    printf("5 * 3 = %d\n", multiply(5, 3));
    printf("5 / 3 = %.2f\n", divide(5, 3));

    return 0;
}
```

编译命令:

```
gcc main.c math_utils.c -o calculator
./calculator
```

Day 18: GNU 工具链

18.1 基本知识点

- GCC 编译器的使用
 - 编译选项: `-o`、`-c`、`-g`、`-Wall`、`-std`
 - 多文件编译
- GDB 调试器基础
 - 启动调试: `gdb <program>`
 - 常用命令: `break`、`run`、`next`、`step`、`print`、`continue`、`quit`
 - 查看变量和调用栈
- Make 工具基础
 - Makefile 的基本结构
 - 目标、依赖、命令
 - 变量的使用

18.2 学习资料

- GCC 官方文档: [GCC Manual](#)
- GDB 官方文档: [GDB Manual](#)
- 廖雪峰的官方网站: [Makefile 教程](#)

18.3 应知应会 Checklist

- ☐ 能使用 GCC 编译单文件和多文件程序
- ☐ 掌握常用的编译选项
- ☐ 能使用 GDB 进行基本调试
- ☐ 了解 Makefile 的基本语法
- ☐ 能编写简单的 Makefile

18.4 小任务

为 Day 17 的多文件项目编写 Makefile 并使用 GDB 调试:

Makefile:

```
CC = gcc
CFLAGS = -Wall -g

calculator: main.o math_utils.o
    $(CC) $(CFLAGS) -o calculator main.o math_utils.o

main.o: main.c math_utils.h
    $(CC) $(CFLAGS) -c main.c

math_utils.o: math_utils.c math_utils.h
    $(CC) $(CFLAGS) -c math_utils.c

clean:
    rm -f *.o calculator
```

编译:

```
make
```

调试:

```
gdb ./calculator
(gdb) break main
(gdb) run
(gdb) next
(gdb) print a
(gdb) quit
```

Day 19: Makefile 与 CMake

19.1 基本知识点

- Makefile 进阶
 - 自动变量: `$$`、`$(<)`、`$(^)`
 - 模式规则
 - 伪目标 (`.PHONY`)
- CMake 基础
 - CMakeLists.txt 文件结构

- 基本命令： `cmake_minimum_required`、 `project`、 `add_executable`
- 跨平台构建
- 生成与编译

19.2 学习资料

- CMake 官方教程：[CMake Tutorial](#)
- 菜鸟教程：[CMake 教程](#)

19.3 应知应会 Checklist

- ☐ 理解 Makefile 的高级特性
- ☐ 能编写规范的 Makefile
- ☐ 了解 CMake 的优势
- ☐ 能编写基本的 CMakeLists.txt
- ☐ 能使用 CMake 构建项目

19.4 小任务

为学生成绩管理系统创建 CMake 项目：

CMakeLists.txt：

```
cmake_minimum_required(VERSION 3.10)
project(StudentManagement)

set(CMAKE_C_STANDARD 11)

# 添加可执行文件
add_executable(student_system
    main.c
    student.c
    file_utils.c
)

# 设置编译选项
target_compile_options(student_system PRIVATE -Wall -Wextra)
```

构建项目：

```
mkdir build
cd build
cmake ..
make
./student_system
```

Day 20: 综合项目① - 学生成绩管理系统

20.1 项目需求

开发一个完整的学生成绩管理系统，要求：

1. **数据结构**：使用结构体存储学生信息（姓名、学号、成绩）
2. **功能模块**：
 - 添加学生
 - 删除学生
 - 查询学生（按学号或姓名）
 - 修改成绩
 - 显示所有学生
 - 保存到文件
 - 从文件加载
3. **模块化设计**：分文件实现，使用头文件
4. **构建系统**：使用 CMake 管理项目

20.2 项目结构

```
StudentManagement/
├── CMakeLists.txt
├── include/
│   ├── student.h
│   └── file_utils.h
├── src/
│   ├── main.c
│   ├── student.c
│   └── file_utils.c
└── data/
    └── students.txt
```

✓ 20.3 实现要点

- ☐ 定义学生结构体和管理结构
- ☐ 实现增删改查功能
- ☐ 实现文件保存和加载
- ☐ 提供用户交互菜单
- ☐ 进行错误处理
- ☐ 编写 CMakeLists.txt

20.4 参考代码框架

include/student.h:

```
#ifndef STUDENT_H
#define STUDENT_H

#define MAX_NAME 50
#define MAX_STUDENTS 100

typedef struct {
    char name[MAX_NAME];
    int id;
    float score;
} Student;

typedef struct {
    Student students[MAX_STUDENTS];
    int count;
} StudentManager;

void initManager(StudentManager *mgr);
int addStudent(StudentManager *mgr, const char *name, int id, float score);
int deleteStudent(StudentManager *mgr, int id);
Student* findStudent(StudentManager *mgr, int id);
void displayAllStudents(StudentManager *mgr);
int updateScore(StudentManager *mgr, int id, float newScore);

#endif
```

include/file_utils.h:

```
#ifndef FILE_UTILS_H
#define FILE_UTILS_H

#include "student.h"

int saveToFile(StudentManager *mgr, const char *filename);
int loadFromFile(StudentManager *mgr, const char *filename);

#endif
```

src/main.c:

```
#include <stdio.h>
#include "student.h"
#include "file_utils.h"

void showMenu() {
    printf("\n==== 学生成绩管理系统 =====\n");
    printf("1. 添加学生\n");
    printf("2. 删除学生\n");
    printf("3. 查询学生\n");
    printf("4. 修改成绩\n");
    printf("5. 显示所有学生\n");
    printf("6. 保存到文件\n");
    printf("7. 从文件加载\n");
    printf("0. 退出\n");
    printf("请选择: ");
}

int main() {
    StudentManager mgr;
    initManager(&mgr);

    int choice;
    char name[MAX_NAME];
    int id;
    float score;

    while(1) {
        showMenu();
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("请输入姓名: ");
                scanf("%s", name);
                printf("请输入学号: ");
                scanf("%d", &id);
                printf("请输入成绩: ");
                scanf("%f", &score);
                addStudent(&mgr, name, id, score);
                break;
            case 2:
                printf("请输入要删除的学号: ");
                scanf("%d", &id);
```

```

        deleteStudent(&mgr, id);
        break;
    case 3:
        printf("请输入要查询的学号: ");
        scanf("%d", &id);
        Student *s = findStudent(&mgr, id);
        if(s) {
            printf("姓名: %s, 学号: %d, 成绩: %.2f\n",
                s->name, s->id, s->score);
        } else {
            printf("未找到该学生\n");
        }
        break;
    case 4:
        printf("请输入学号: ");
        scanf("%d", &id);
        printf("请输入新成绩: ");
        scanf("%f", &score);
        updateScore(&mgr, id, score);
        break;
    case 5:
        displayAllStudents(&mgr);
        break;
    case 6:
        saveToFile(&mgr, "data/students.txt");
        break;
    case 7:
        loadFromFile(&mgr, "data/students.txt");
        break;
    case 0:
        printf("再见! \n");
        return 0;
    default:
        printf("无效选择\n");
}

}

return 0;
}

```

Day 21: 综合项目② - 控制台 Flappy Bird (扩展挑战)

21.1 项目概述

实现一个控制台版本的 Flappy Bird 小游戏，包含以下要素：

- 小鸟的跳跃控制
- 障碍物的生成与移动
- 碰撞检测
- 分数统计
- 游戏结束判定

21.2 实现要点

- 使用字符界面绘制游戏场景
- 实现简单的物理引擎（重力、跳跃）
- 使用定时器控制游戏速度
- 处理用户输入（空格键跳跃）
- 实现障碍物的随机生成

21.3 技术要点

- ☐ 控制台输出控制（清屏、光标定位）
- ☐ 非阻塞输入处理
- ☐ 游戏主循环设计
- ☐ 碰撞检测算法
- ☐ 分数系统实现

21.4 参考代码框架

game.h:

```

#ifndef GAME_H
#define GAME_H

#define WIDTH 50
#define HEIGHT 20
#define BIRD_X 10

typedef struct {
    int y;           // 小鸟的 Y 坐标
    int velocity;    // 小鸟的速度
} Bird;

typedef struct {
    int x;           // 障碍物的 X 坐标
    int gap_y;       // 缺口的 Y 坐标
    int gap_size;    // 缺口大小
} Obstacle;

typedef struct {
    Bird bird;
    Obstacle obstacles[3];
    int score;
    int game_over;
} Game;

void initGame(Game *game);
void updateGame(Game *game);
void drawGame(Game *game);
int checkCollision(Game *game);
void jump(Bird *bird);

#endif

```

main.c (简化版):

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "game.h"

#ifdef _WIN32
#include <conio.h>
#include <windows.h>
#else
#include <unistd.h>
#include <termios.h>
#endif

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

int main() {
    Game game;
    initGame(&game);

    srand(time(NULL));

    printf("Flappy Bird - 按空格键跳跃, 按 Q 退出\n");
    printf("按任意键开始...\n");
    getchar();

    while(!game.game_over) {
        clearScreen();
        updateGame(&game);
        drawGame(&game);

        // 检查用户输入
#ifdef _WIN32
        if(_kbhit()) {
            char ch = _getch();
            if(ch == ' ') jump(&game.bird);
            if(ch == 'q' || ch == 'Q') break;
        }

```

```

#endif

    // 检查碰撞
    if(checkCollision(&game)) {
        game.game_over = 1;
    }

#ifdef _WIN32
    Sleep(100); // 100ms
#else
    usleep(100000);
#endif

    }

    clearScreen();
    printf("游戏结束! 得分: %d\n", game.score);

    return 0;
}

```

提示：完整实现需要补充 `game.c` 中的各个函数，处理小鸟的物理运动、障碍物的生成与移动、碰撞检测等逻辑。

五、学习建议与总结

学习建议

1. **每天坚持编码：**理论学习后务必动手实践，将代码运行起来。
2. **理解而非死记：**重点理解指针、内存管理等核心概念，而不是死记语法。
3. **善用调试工具：**学会使用 GDB 调试，能极大提升解决问题的能力。
4. **多阅读代码：**参考开源项目和优秀代码，学习编程风格和设计思想。
5. **循序渐进：**从简单到复杂，逐步掌握 C 语言的各个特性。
6. **保持耐心：**C 语言学习曲线较陡峭，遇到困难是正常的，保持耐心和毅力。

进阶方向

完成本教程后，可以继续学习：

- **数据结构与算法：**链表、栈、队列、树、图等

- **操作系统编程**：进程、线程、进程间通信
- **网络编程**：Socket 编程、TCP/IP 协议
- **嵌入式开发**：单片机编程、实时操作系统
- **系统编程**：Linux 系统编程、驱动开发

总结

通过三周的系统学习，你应该已经掌握了 C 语言的核心知识，能够编写结构化的程序，理解内存管理和指针的使用，掌握模块化编程和工程化构建。

C 语言是一门强大而经典的编程语言，是深入理解计算机系统的重要工具。希望这份教程能够帮助你顺利入门 C 语言，为后续的学习和项目开发打下坚实的基础。

加油！编程之路，始于足下！ 💪